

MATHEMATICS FOR

AI ENGINEERING

Interview Questions & Answers

Prepared for
Sameeksha

Covering: Linear Algebra | Calculus & Optimization
Probability & Statistics | Information Theory

19 Questions | Deep Theory | Interview Ready

TABLE OF CONTENTS

01. Linear Algebra 5 Questions

02. Calculus & Optimization 5 Questions

03. Probability & Statistics 5 Questions

04. Information Theory 2 Questions

01 Linear Algebra

Q
1

What is an eigenvector and eigenvalue? Why are they important in ML?

Answer:

An eigenvector of a matrix A is a non-zero vector v such that $Av = \lambda v$, where λ is the corresponding eigenvalue — a scalar. Geometrically, the matrix only stretches or shrinks the eigenvector, without rotating it. In ML, eigenvalues and eigenvectors are foundational to PCA (Principal Component Analysis), where eigenvectors of the covariance matrix give directions of maximum variance. They also appear in spectral clustering, kernel methods, and understanding convergence in optimization algorithms like gradient descent.

Tip: Key insight: PCA finds the top- k eigenvectors of the covariance matrix to reduce dimensionality while preserving maximum variance.

Q
2

What is Singular Value Decomposition (SVD) and where is it used?

Answer:

SVD decomposes any matrix A ($m \times n$) into $A = U * \Sigma * V^T$, where U is an $m \times m$ orthogonal matrix (left singular vectors), Σ is an $m \times n$ diagonal matrix of singular values (non-negative), and V^T is an $n \times n$ orthogonal matrix (right singular vectors). Unlike eigendecomposition, SVD works for any matrix — not just square ones. In AI/ML, SVD powers dimensionality reduction (truncated SVD / LSA in NLP), collaborative filtering in recommendation systems, image compression, and pseudo-inverse computation for solving least squares problems.

Tip: SVD is the generalization of eigendecomposition — singular values are the square roots of eigenvalues of $A^T A$.

Q
3

What does it mean for a matrix to be positive semi-definite (PSD)?

Answer:

A symmetric matrix M is positive semi-definite if for all vectors z , $z^T M z \geq 0$. Equivalently, all eigenvalues of M are non-negative. A matrix is positive definite (PD) if $z^T M z > 0$ for all non-zero z (all eigenvalues strictly positive). PSD matrices appear frequently in ML: covariance matrices are always PSD, kernel matrices (Gram matrices) must be PSD for valid kernels, and the Hessian of a convex function is PSD everywhere.

Tip: Covariance matrices are always PSD — this ensures variances are non-negative and the geometry of data is valid.

Q
4

Explain matrix rank and its significance in ML.

Answer:

The rank of a matrix is the dimension of its column space (or equivalently, row space) — the maximum number of linearly independent rows or columns. A matrix is full rank if rank equals $\min(m, n)$. In ML, a low-rank weight matrix in a neural network can be factorized to reduce parameters (used in LoRA for LLMs). Rank deficiency causes issues in solving linear systems — the matrix becomes non-invertible. In data, rank represents the intrinsic dimensionality: if the data matrix has low rank, features are linearly correlated.

Tip: Low-rank approximations (via SVD) are widely used in model compression and fine-tuning large language models (LoRA).

Q
5

What is the dot product and why is it central to neural networks?

Answer:

The dot product of two vectors a and b is $a \cdot b = \sum(a_i * b_i) = \|a\| \|b\| \cos(\theta)$, where θ is the angle between them. It measures how aligned two vectors are. In neural networks, every linear layer computes $y = Wx + b$, which is fundamentally a series of dot products. Attention mechanisms (Transformers) compute similarity scores via QK^T , which are dot products between query and key vectors. Cosine similarity, derived from the dot product, is used in embedding search, NLP, and recommendation systems.

Tip: The entire forward pass of a transformer is built on dot products — understanding this helps debug attention and embedding behavior.

02 Calculus & Optimization

Q
1

What is the chain rule and how is it used in backpropagation?

Answer:

The chain rule states that for composite functions, $d/dx[f(g(x))] = f'(g(x)) * g'(x)$. In backpropagation, a neural network is a nested composition of functions (layers). The gradient of the loss with respect to any weight is computed by chaining partial derivatives from the output back to that weight: $dL/dW = dL/da * da/dz * dz/dW$. This is done efficiently using dynamic programming — intermediate gradients are cached and reused. Without the chain rule, training deep networks would be computationally intractable.

Tip: *Backprop is just the chain rule applied systematically — understanding it prevents common bugs like vanishing/exploding gradients.*

Q
2

What is the gradient, and how does gradient descent minimize a loss function?

Answer:

The gradient of a scalar function $f(x)$ is a vector of partial derivatives $[df/dx_1, df/dx_2, \dots, df/dx_n]$ pointing in the direction of steepest increase. Gradient descent updates parameters by moving in the opposite direction: $\theta = \theta - \text{learning_rate} * \text{gradient}(L)$. Stochastic Gradient Descent (SGD) uses a mini-batch to approximate the full gradient, making it computationally feasible for large datasets. Variants like Adam, RMSProp, and Adagrad adapt the learning rate per parameter for faster convergence.

Tip: *Adam = momentum + adaptive learning rates. It is the default optimizer for most deep learning tasks due to its robustness.*

Q
3

What is a Jacobian matrix? How does it relate to backpropagation?

Answer:

The Jacobian of a vector-valued function $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ is an $m \times n$ matrix where $J_{ij} = df_i/dx_j$. It generalizes the gradient to vector outputs. In backpropagation, when a layer maps a vector to another vector, the backward pass involves multiplying by the Jacobian (or its transpose). For efficiency, we never explicitly compute the full Jacobian — instead, we compute Jacobian-vector products (JVPs) or vector-Jacobian products (VJPs), which is what autograd frameworks like PyTorch implement.

Tip: *Modern autograd systems (PyTorch, JAX) compute VJPs implicitly — they never materialize the full Jacobian matrix.*

Q
4**What is convexity? Why does it matter in optimization?****Answer:**

A function f is convex if for any two points x, y and t in $[0,1]$: $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$. Geometrically, the line segment between any two points lies above the function. Convex problems are desirable because any local minimum is the global minimum, and gradient descent is guaranteed to converge. Most deep learning loss landscapes are non-convex due to multiple layers and nonlinear activations. However, certain parts of ML (logistic regression, SVMs with convex kernels) are convex and have well-understood convergence guarantees.

Tip: *Logistic regression is convex; neural networks are not. This is why NNs may get stuck in local minima.*

Q
5**What is the difference between first-order and second-order optimization methods?****Answer:**

First-order methods use only gradient information (first derivatives): SGD, Adam, RMSProp. They are computationally cheap — $O(n)$ per step — making them practical for millions of parameters. Second-order methods use the Hessian (second derivatives) or approximations of it. Newton's method uses $H^{-1} * \text{gradient}$ for updates, offering faster convergence near optima but at $O(n^2)$ or $O(n^3)$ cost per step. L-BFGS approximates the inverse Hessian using gradient history and is used for smaller-scale ML problems. Second-order methods are rarely used in deep learning due to computational cost.

Tip: *Adam approximates second-order behavior using gradient squared history — it is a practical compromise.*

03 Probability & Statistics

Q
1

What is Bayes' Theorem and how is it used in ML?

Answer:

Bayes' theorem: $P(A|B) = P(B|A) * P(A) / P(B)$. It updates our belief about A given new evidence B. In ML, it underlies Naive Bayes classifiers (assuming feature independence), Bayesian inference for posterior estimation, and Bayesian Neural Networks which maintain distributions over weights instead of point estimates. In broader terms, maximum a posteriori (MAP) estimation is Bayesian: it maximizes $P(\theta|data) = P(data|\theta) * P(\theta)$. L2 regularization corresponds exactly to a Gaussian prior on weights in the MAP framework.

Tip: L2 regularization = Gaussian prior on weights (MAP estimation). L1 regularization = Laplace prior.

Q
2

What is the difference between MLE and MAP estimation?

Answer:

Maximum Likelihood Estimation (MLE) finds parameters that maximize $P(data | \theta)$ — it finds the most likely explanation of the data given the parameters, with no prior assumption. MAP (Maximum A Posteriori) maximizes $P(\theta | data) = P(data | \theta) * P(\theta)$, incorporating a prior belief about parameters. MAP is equivalent to MLE with regularization: the prior acts as a regularizer. With a Gaussian prior, MAP gives L2-regularized MLE. As data grows large, the prior becomes negligible and MAP converges to MLE.

Tip: With enough data, MLE and MAP agree. With limited data, MAP is better as the prior prevents overfitting.

Q
3

What is cross-entropy loss and why is it preferred over MSE for classification?

Answer:

Cross-entropy between true distribution p and predicted distribution q is: $H(p,q) = -\sum(p_i * \log(q_i))$. For binary classification, it becomes: $-[y*\log(p) + (1-y)*\log(1-p)]$. It is preferred over MSE for classification because: (1) it has steeper gradients when predictions are very wrong, making training faster; (2) it directly models the log-likelihood of the predicted probability under the true labels; (3) combined with softmax, the gradient simplifies to (predicted - true), making backprop clean and numerically stable.

Tip: Softmax + Cross-Entropy gradient = $(\hat{y} - y)$. This clean gradient is why this combination dominates classification tasks.

Q
4

Explain the bias-variance tradeoff.

Answer:

The total expected prediction error decomposes as: $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$. Bias measures how far the average prediction is from the true value — high bias means underfitting (model too simple). Variance measures how much predictions fluctuate across different training sets — high variance means overfitting (model too complex). The tradeoff: reducing bias often increases variance and vice versa. Regularization, dropout, ensemble methods, and cross-validation are tools to manage this balance. Deep learning somewhat challenges this — overparameterized models can simultaneously have low bias and low variance (double descent phenomenon).

Tip: Ensemble methods (bagging) reduce variance; boosting reduces bias. Regularization increases bias but reduces variance.

Q
5

What is the Central Limit Theorem (CLT) and why is it relevant to ML?

Answer:

The CLT states that the sum (or mean) of a large number of i.i.d. random variables, regardless of their original distribution, converges to a Normal distribution as the sample size increases. In ML, CLT justifies: using Gaussian distributions to model noise and errors, the effectiveness of mini-batch gradient descent (mean gradient over a batch approximates the true gradient), hypothesis testing and confidence intervals in model evaluation, and weight initialization strategies (e.g., Xavier/He initialization) which use Gaussian distributions.

Tip: Mini-batch gradients are approximately Gaussian by CLT — this is why stochastic optimization works despite noisy estimates.

04 Information Theory

Q
1

What is entropy in information theory and its relation to ML?

Answer:

Shannon entropy $H(X) = -\sum(p(x) * \log_2(p(x)))$ measures the average uncertainty or information content of a random variable. High entropy means high unpredictability (uniform distribution). Zero entropy means certainty (deterministic). In ML: cross-entropy loss is derived from entropy, decision trees use entropy (or Gini impurity) to select splits that maximize information gain, variational autoencoders use KL divergence (related to entropy) to regularize the latent space, and the mutual information between features and labels guides feature selection.

Tip: Entropy is maximized for uniform distributions and minimized (zero) for deterministic ones.

Q
2

What is KL Divergence and how is it used in VAEs?

Answer:

Kullback-Leibler divergence $KL(P \parallel Q) = \sum(P(x) * \log(P(x)/Q(x)))$ measures how different distribution P is from reference distribution Q. It is asymmetric: $KL(P \parallel Q) \neq KL(Q \parallel P)$. $KL = 0$ iff $P = Q$. In Variational Autoencoders (VAEs), the loss has two terms: reconstruction loss (cross-entropy or MSE) + $\beta * KL(q(z|x) \parallel p(z))$, where the KL term forces the learned posterior to stay close to the prior (typically $N(0, I)$). This regularizes the latent space, enabling smooth interpolation and generation.

Tip: VAE loss = Reconstruction loss + KL divergence. The KL term prevents posterior collapse and enables generation.